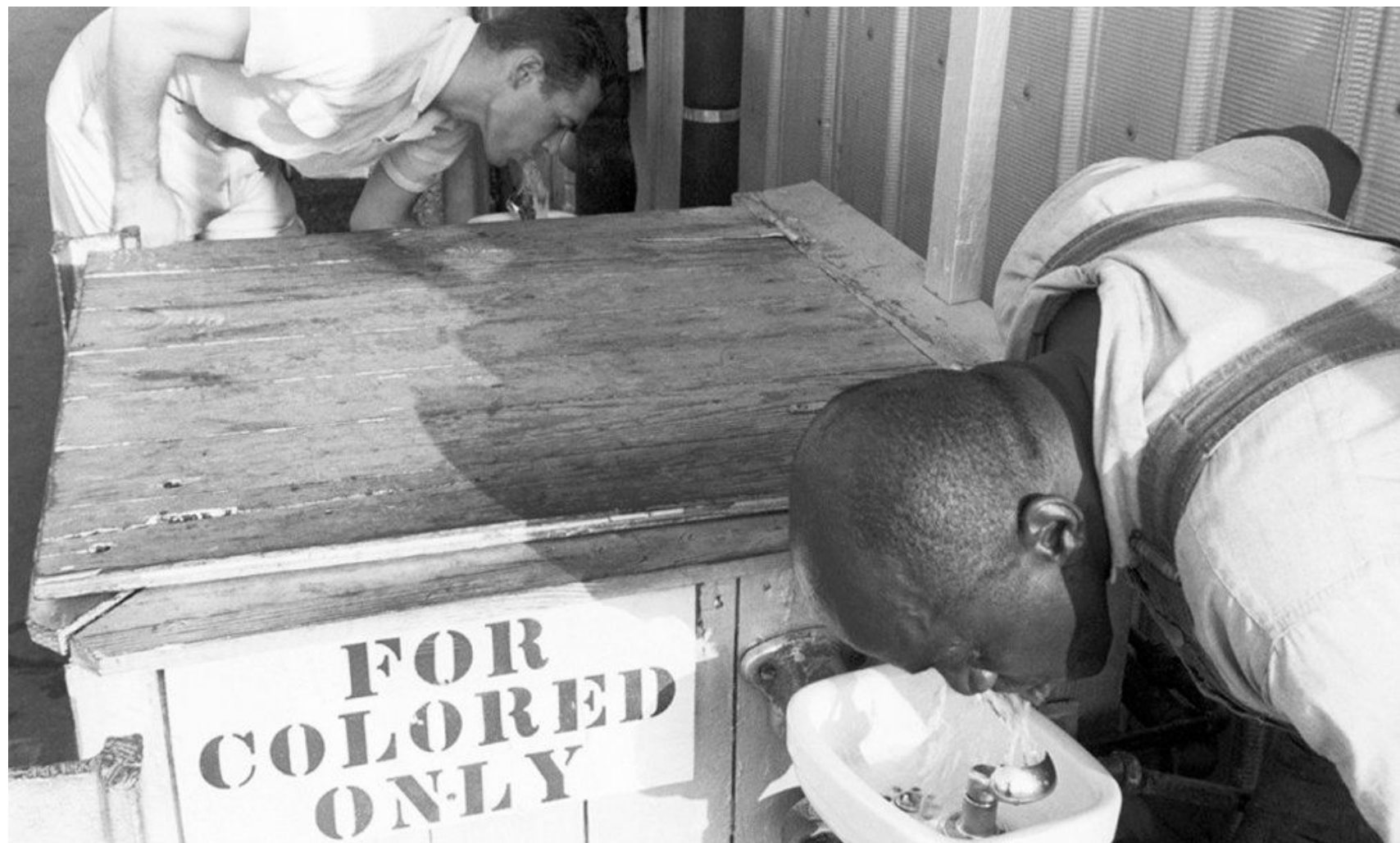


INTERFACE SEGREGATION

By João Ribeiro
23 / 03 / 2023





```
public class ComeToAcademia implements Transportes{  
    @Override  
    public void buyPass() {  
  
    }  
  
    @Override  
    public void catchStrike() {  
  
    }  
  
    @Override  
    public void takeBus() {  
  
    }  
  
    @Override  
    public void takeMetro() {  
  
    }  
  
    @Override  
    public void takeTrain() {  
  
    }  
  
    @Override  
    public void takeboat() {  
  
    }  
}
```



```
public interface Transportes {  
  
    void buyPass();  
  
    void catchStrike();  
  
    void takeBus();  
  
    void takeMetro();  
  
    void takeTrain();  
  
    void takeboat();  
  
}
```

```
public class ComeToAcademia implements Transportes{
    @Override
    public void buyPass() {

    }

    @Override
    public void catchStrike() {

    }

    @Override
    public void takeBus() {

    }

    @Override
    public void takeMetro() {

    }

    @Override
    public void takeTrain() {
        throw new UnsupportedOperationException("No needed");
    }

    @Override
    public void takeboat() {
        throw new UnsupportedOperationException("No needed");
    }
}
```

Bad !!!

ROBERT CECIL MARTIN

Born 5 December 1952

Colloquially called "Uncle Bob"

Is a American software engineer, instructor, and author.

He is most recognized for promoting many software design principles and for being an author and signatory of the influential

- Agile Manifesto.
- SOLID Principles



SOLID: THE FIRST 5 PRINCIPLES OF OBJECT ORIENTED DESIGN

- These principles establish practices that lend to developing software with considerations for maintaining and extending as the project grows;

- **S** - Single-responsibility Principle
- **O** - Open-closed Principle
- **L** - Liskov Substitution Principle
- **I** - Interface Segregation Principle
- **D** - Dependency Inversion Principle

A class should have one and only one reason to change, meaning that a class should have only one job.

Objects or entities should be open for extension but closed for modification.



```
public interface Transport {  
  
    void buyPass();  
  
    void catchStrike();  
  
    void takeBus();  
  
}
```



```
public interface Lisbon extends Transport {  
  
    void takeMetro();  
  
}
```

```
public interface SouthSide extends Transport {  
  
    void takeTrain();  
    void takeboat();  
  
}
```



```
public class Lisboa implements Lisbon{
```

```
    @Override  
    public void buyPass() {  
  
    }  
}
```

```
    @Override  
    public void catchStrike() {  
  
    }  
}
```

```
    @Override  
    public void takeBus() {  
  
    }  
}
```

```
    @Override  
    public void takeMetro() {  
  
    }  
}
```

```
}
```



```
public class Mitra implements SouthSide{
```

```
    @Override  
    public void buyPass() {  
  
    }  
}
```

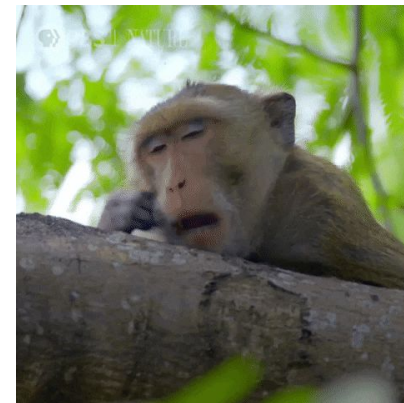
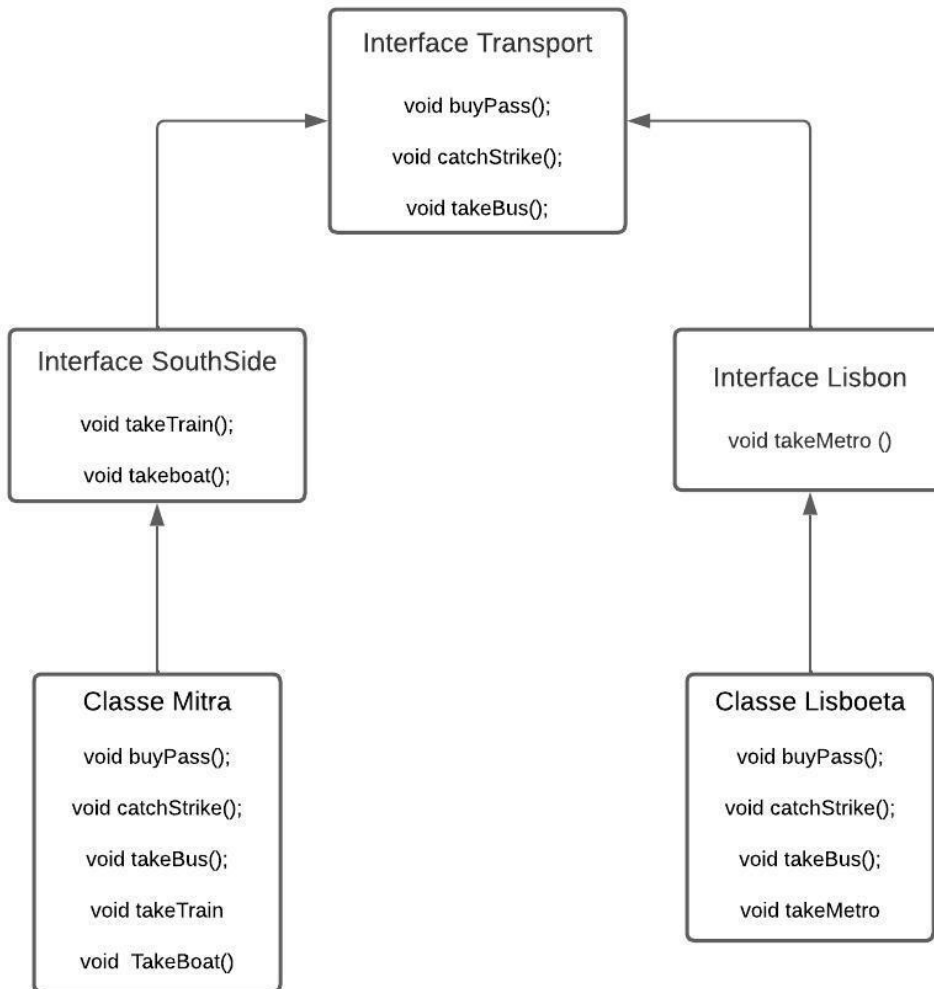
```
    @Override  
    public void catchStrike() {  
  
    }  
}
```

```
    @Override  
    public void takeBus() {  
  
    }  
}
```

```
    @Override  
    public void takeTrain() {  
  
    }  
}
```

```
    @Override  
    public void takeboat() {  
  
    }  
}
```

```
}
```



THANK YOU!

